

Marketing Campaign Prediction Report

2024-05-14

Table of content

- 1.1 Introduction
- 1.2 Data Cleaning and Wrangling
- 1.3 Exploratory Data Analysis
- 1.4 Feature Engineering
- 1.5 Model Prediction
 - 1.5.1 Decision Tree
 - 1.5.2 Logistics Regression
 - 1.5.3 K-Nearest Neighbor
 - 1.5.4 Clustering
- 1.6 Conclusion
- 1.7 Conclusion

Refences

1.1 Introduction

The significance of a successful marketing campaign in the contemporary digitized era cannot be underestimated, given its numerous advantages. While certain enterprises are witnessing notable growth as a result of their marketing efforts, others encounter various obstacles in maintaining competitiveness, often stemming from a lack of effective marketing strategies. As emphasized by Lee and Cho (2019), the importance of marketing campaigns has escalated for all organizations, necessitating the implementation of targeted approaches to engage potential clientele. Essentially, with the rising prevalence of online distribution channels, organizations are compelled to devise marketing tactics that set them apart from their rivals (Wilkes et al., 2021).However, predicting the success of marketing campaigns is a crucial task for businesses seeking to optimize their marketing efforts and allocate resources effectively. In essence, in light of the increasing prevalence of online delivery platforms, organizations are required to develop marketing strategies that distinguish themselves from their competitors. This project gives a deep insight on customer response (yes/no) for a campaign offer based on some particular attributes using machine learning.

The dataset pertaining to the marketing campaign was retrieved from a MySql database. The following section offers a comprehensive outline of the project, encompassing data extraction, cleansing, and manipulation, preprocessing, feature enhancement, and model forecasting. The necessary library was imported to guarantee the seamless implementation of codes. To put it differently, utilizing the suggestions derived from the model's outcomes can assist organizations in enhancing their marketing strategies and attaining enduring growth within the contemporary competitive market landscape.

```
library(tidyverse)

## Warning: package 'tidyverse' was built under R version 4.2.3

## Warning: package 'ggplot2' was built under R version 4.2.3

## Warning: package 'tibble' was built under R version 4.2.3

## Warning: package 'tidyr' was built under R version 4.2.3

## Warning: package 'readr' was built under R version 4.2.3
```

```
## Warning: package 'purrr' was built under R version 4.2.3
```

```
## Warning: package 'dplyr' was built under R version 4.2.3
```

```
## Warning: package 'stringr' was built under R version 4.2.3
```

```
## Warning: package 'forcats' was built under R version 4.2.3
```

```
## Warning: package 'lubridate' was built under R version 4.2.3
```

```
## — Attaching core tidyverse packages ————— tidyverse 2.0.0 —
## ✓ dplyr      1.1.4    ✓ readr      2.1.5
## ✓ forcats    1.0.0    ✓ stringr    1.5.1
## ✓ ggplot2    3.4.4    ✓ tibble     3.2.1
## ✓ lubridate  1.9.3    ✓ tidyr      1.3.1
## ✓ purrr      1.0.2
## — Conflicts ————— tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## ⓘ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tree)
```

```
## Warning: package 'tree' was built under R version 4.2.3
```

```
library(rpart)
```

```
## Warning: package 'rpart' was built under R version 4.2.3
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.2.3
```

```
## Loading required package: lattice
```

```
## Warning: package 'lattice' was built under R version 4.2.3
```

```
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##   lift
```

```
library(stats)
library(class)
```

```
## Warning: package 'class' was built under R version 4.2.3
```

```
library(dendextend)
```

```
## Warning: package 'dendextend' was built under R version 4.2.3
```

```
##
## -----
## Welcome to dendextend version 1.17.1
## Type citation('dendextend') for how to cite the package.
##
## Type browseVignettes(package = 'dendextend') for the package vignette.
## The github page is: https://github.com/talgalili/dendextend/
##
## Suggestions and bug-reports can be submitted at: https://github.com/talgalili/dendextend/issues
## You may ask questions at stackoverflow, use the r and dendextend tags:
##   https://stackoverflow.com/questions/tagged/dendextend
##
## To suppress this message use: suppressPackageStartupMessages(library(dendextend))
## -----
##
##
## Attaching package: 'dendextend'
##
## The following object is masked from 'package:rpart':
##
##   prune
##
## The following object is masked from 'package:stats':
##
##   cutree
```

```
library(colorspace)
```

```
## Warning: package 'colorspace' was built under R version 4.2.3
```

```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.2.3
```

```
## corrplot 0.92 loaded
```

```
library(pROC)
```

```
## Warning: package 'pROC' was built under R version 4.2.3
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following object is masked from 'package:colorspace':
##
##     coords
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
library(cluster)
```

```
## Warning: package 'cluster' was built under R version 4.2.3
```

```
library(factoextra)
```

```
## Warning: package 'factoextra' was built under R version 4.2.3
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
library(DBI)
library(RMySQL)
```

```
## Warning: package 'RMySQL' was built under R version 4.2.3
```

```
#extraction of the dataset from mysql
```

```
USER <- 'root'
PASSWORD <- 'harkinkunmsey'
HOST <- 'localhost'
DBNAME <- 'world'
```

```
# run queries
```

```
db <- dbConnect(MySQL(), user = USER, password = PASSWORD,
               host = HOST, dbname = DBNAME, port=3306) # connect to database
```

```
clean_data <- dbGetQuery(db, statement = "Select * from world.marketingcampaign")
```

```
dbDisconnect(db)
```

```
## [1] TRUE
```

```
# dimension of the data
dim(clean_data)
```

```
## [1] 22141    10
```

```
#structure of the dataset
str(clean_data)
```

```
## 'data.frame':    22141 obs. of  10 variables:
## $ ID              : int  11000 11001 11002 11003 11004 11005 11006 11007 11008 11009 ...
## $ Year_Birth       : int  1969 1963 1951 1979 1969 1981 1955 1989 1983 1981 ...
## $ Education        : chr   "Graduation" "PhD" "Master" "Graduation" ...
## $ MaritalStatus    : chr   "Together" "Single" "Married" "Single" ...
## $ Income           : int  23228 48918 67381 61825 44078 41967 75261 28691 24072 19414 ...
## $ Recency          : int   71  21  67  56  17  66  17  56  79  32 ...
## $ NumWebPurchases  : int    2  1  2  4  2  1  5  1  1  1 ...
## $ NumStorePurchases: int    3  4  9  8  3  3  5  3  2  3 ...
## $ NumWebVisitsMonth: int    8  4  7  4  5  4  2  8  8  8 ...
## $ Response         : int    0  0  0  0  0  0  1  0  0  0 ...
```

```
# data types of each variables
sapply(clean_data, class)
```

```
##           ID           Year_Birth           Education           MaritalStatus
##      "integer"      "integer"      "character"      "character"
##           Income           Recency NumWebPurchases NumStorePurchases
##      "integer"      "integer"      "integer"      "integer"
## NumWebVisitsMonth           Response
##      "integer"      "integer"
```

```
# brief overview of the dataset
head(clean_data)
```

ID	Year_Birth	Education	MaritalStatus	Inco...	Rece...	NumWebPurcha...	NumStorePurchases
<int>	<int>	<chr>	<chr>	<int>	<int>	<int>	<int>
1 11000	1969	Graduation	Together	23228	71	2	3
2 11001	1963	PhD	Single	48918	21	1	4
3 11002	1951	Master	Married	67381	67	2	9
4 11003	1979	Graduation	Single	61825	56	4	8
5 11004	1969	Graduation	Married	44078	17	2	3
6 11005	1981	Graduation	Single	41967	66	1	3

6 rows | 1-9 of 11 columns

```
# descriptive analysis of the dataset
summary(clean_data)
```

```
##      ID      Year_Birth      Education      MaritalStatus
## Min.   :11000   Min.   :1893   Length:22141   Length:22141
## 1st Qu.:16592   1st Qu.:1959   Class :character   Class :character
## Median :22197   Median :1970   Mode  :character   Mode  :character
## Mean   :22198   Mean   :1969
## 3rd Qu.:27799   3rd Qu.:1978
## Max.   :33399   Max.   :1996
##      Income      Recency      NumWebPurchases      NumStorePurchases
## Min.    : 1730   Min.    : 0.00   Min.    : 0.000   Min.    : 0.000
## 1st Qu.: 35441   1st Qu.:24.00   1st Qu.: 2.000   1st Qu.: 3.000
## Median : 51529   Median :49.00   Median : 4.000   Median : 5.000
## Mean    : 52514   Mean    :48.78   Mean    : 4.103   Mean    : 5.801
## 3rd Qu.: 68682   3rd Qu.:73.00   3rd Qu.: 6.000   3rd Qu.: 8.000
## Max.    :666666   Max.    :99.00   Max.    :27.000   Max.    :13.000
## NumWebVisitsMonth      Response
## Min.    : 0.000   Min.    :0.0000
## 1st Qu.: 3.000   1st Qu.:0.0000
## Median : 6.000   Median :0.0000
## Mean    : 5.317   Mean    :0.1532
## 3rd Qu.: 7.000   3rd Qu.:0.0000
## Max.    :20.000   Max.    :1.0000
```

```
## Data Cleaning and Wrangling
# check for any missing value
sum(is.na(clean_data))
```

```
## [1] 0
```

```
#checking the response variable
cbind(freq = table(clean_data$Response), per=prop.table(table(clean_data$Response))*100)
```

```
##      freq      per
## 0 18748 84.67549
## 1  3393 15.32451
```

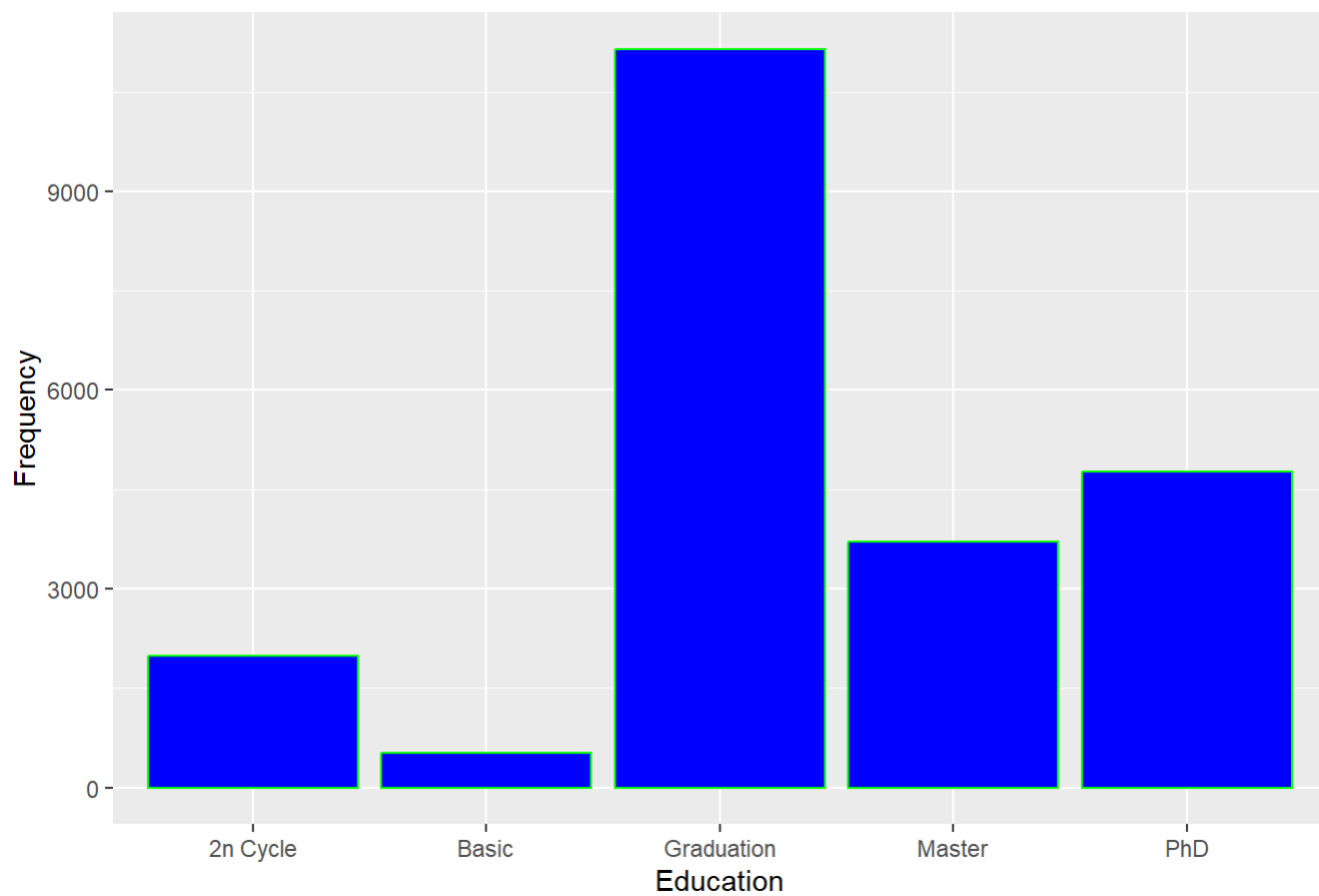
1.2 Data Cleaning and Wrangling

Upon examination of the data, it is crucial to be cognizant of the specific type of dataset one is dealing with, in order to determine the appropriate procedures for data cleansing and manipulation.

The dataset was found to comprise 22,400 entries and 10 variables, with the respondent's income ranging from 1730 to 666666, and an average income of 52,514. The mean duration since the customer's last purchase is 48.89 days, with the most recent purchase dating back to 99 days ago, and an average of 5.314 visits to the company's website in the last month. The average customer age is 1969. Furthermore, the mean number of online purchases is 4.104, and the average number of in-store purchases is 5.785. It was determined that merely 15% of customers responded positively to the recent campaign offer, while the vast majority, 85%, declined the offer.

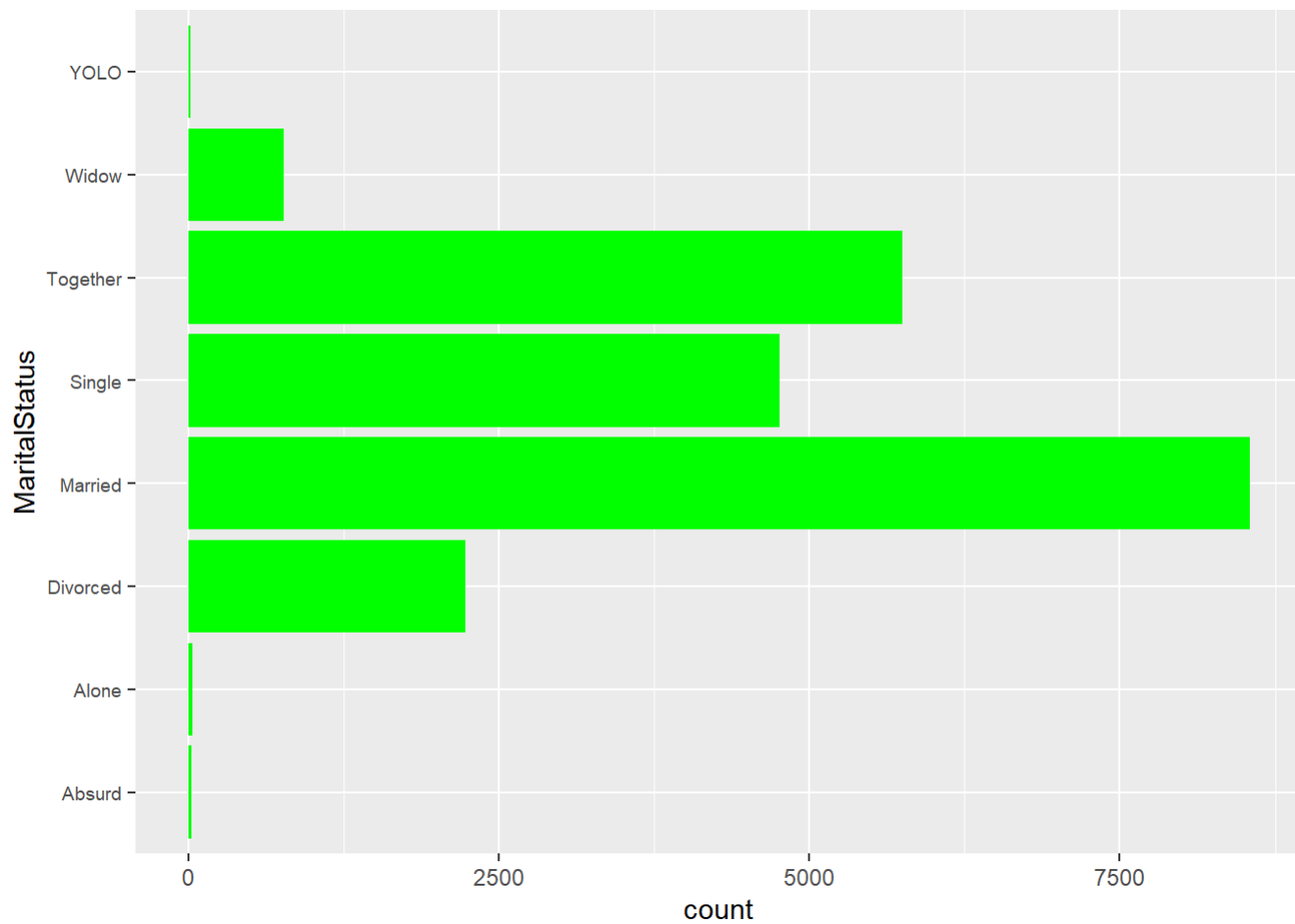
```
#viz EDA
# visualisation of Education of the Respondent
ggplot(clean_data, aes(x = Education)) +
  geom_bar(position = "dodge", color = "Green", fill = "blue")+
  labs(title = "Education of the Respondent",
       x = "Education",
       y = "Frequency",
       fill = "Response")
```

Education of the Respondent



#comparison of Marital Status to those that responded to the campaign

```
ggplot(clean_data) + geom_bar(aes(x=MaritalStatus, fill = Response), fill="green")+coord_flip() +  
  theme(axis.text.y=element_text(size=rel(0.8)))
```



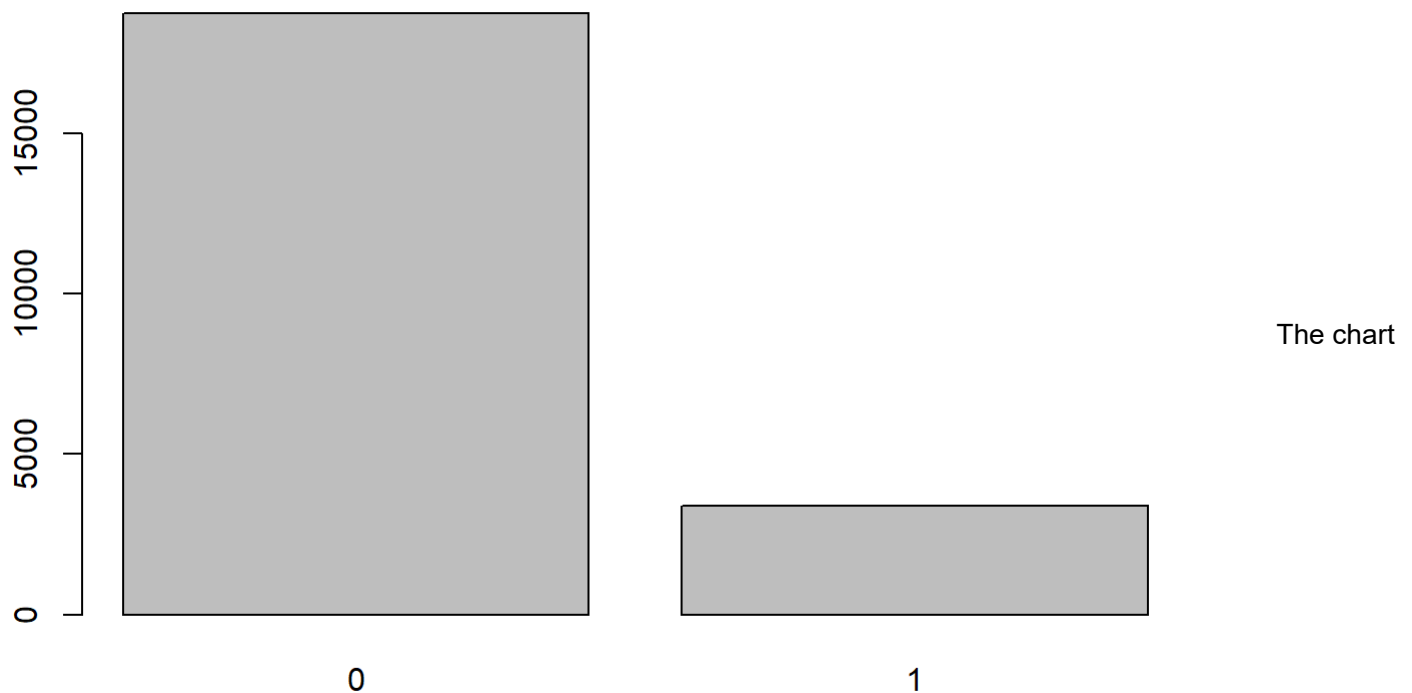
```
# education and campaign response
table(clean_data$Education, clean_data$Response)
```

```
##
##           0    1
## 2n Cycle 1792 191
## Basic    502  16
## Graduation 9558 1592
## Master   3145  572
## PhD      3751 1022
```

```
# Marital_Status and campaign response
table(clean_data$MaritalStatus, clean_data$Response)
```

```
##
##           0    1
## Absurd    20    9
## Alone     23   15
## Divorced 1789  440
## Married  7531 1016
## Single   3653 1111
## Together 5164  584
## Widow    563  208
## YOLO       5   10
```

```
# visualisation of the Response Variable
barplot(table(clean_data$Response))
```

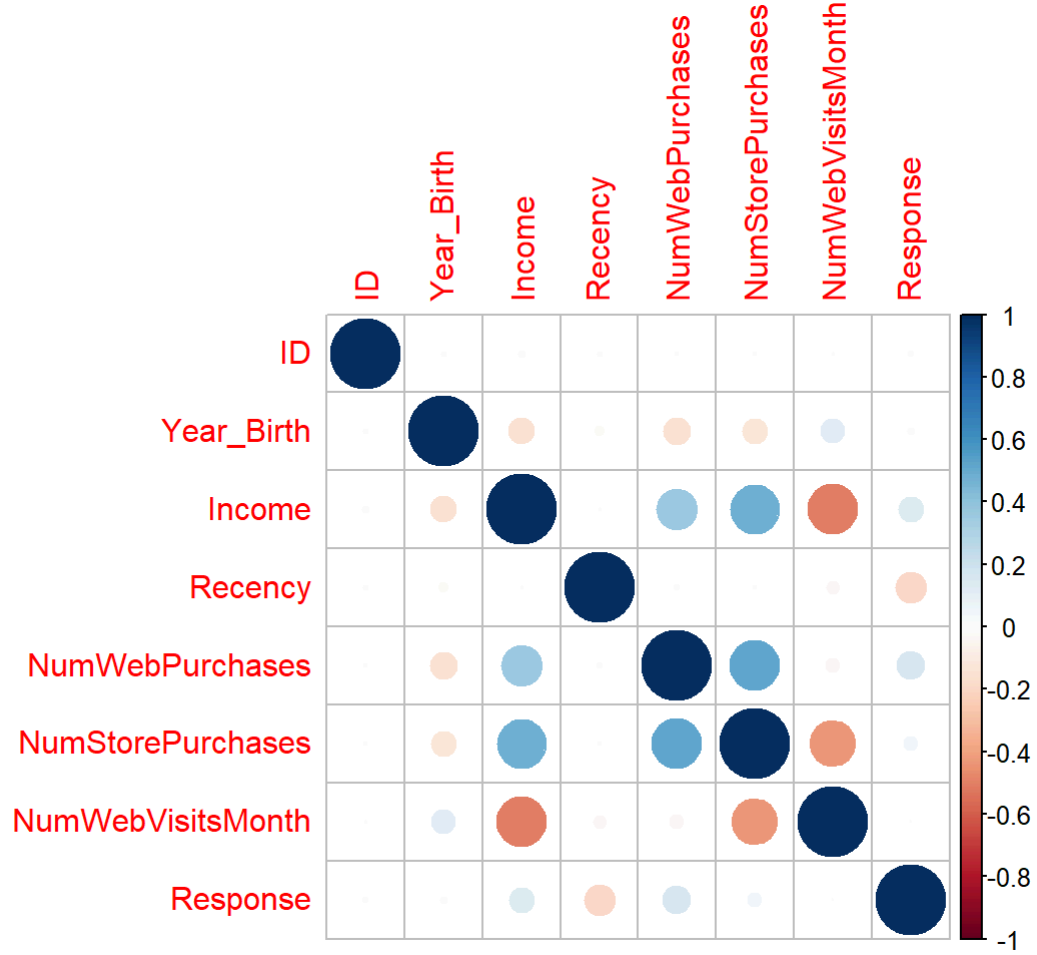


above illustrates the response to marketing campaign offer where It was observed that most of the customers cleaned the offer.

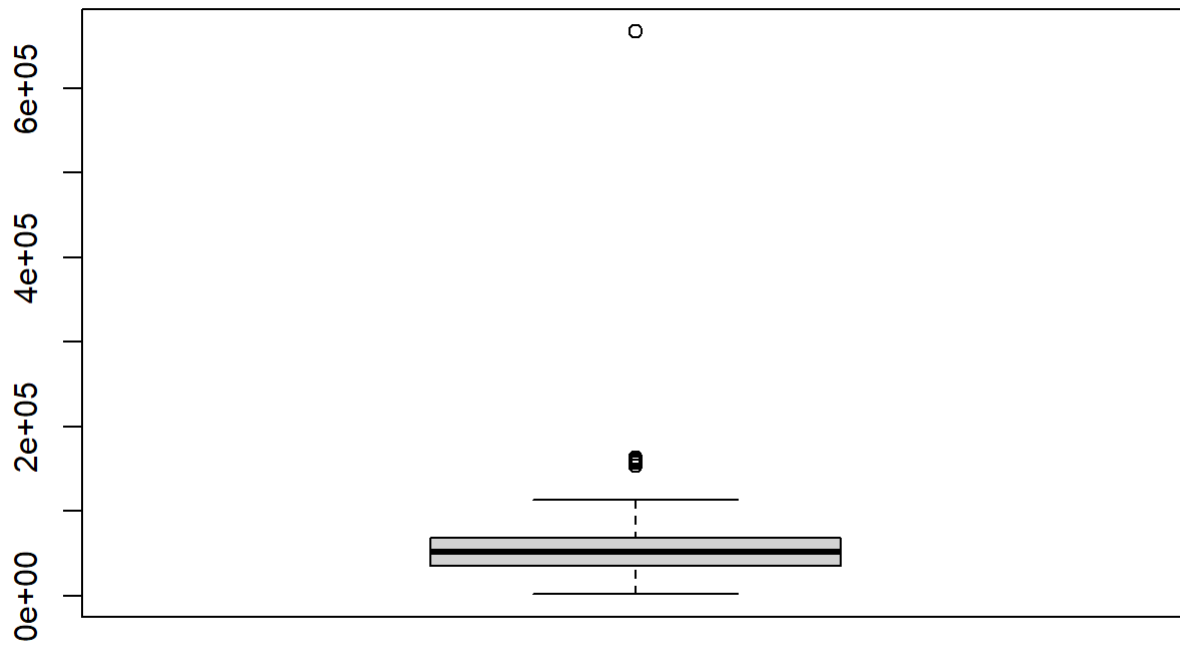
```
# correlation of all the numerical columns to campaign response
correlation <- cor(clean_data[,apply(clean_data, is.numeric)])
correlation
```

```
##              ID   Year_Birth      Income      Recency
## ID          1.000000000 -0.004914787  0.009816702  0.005182121
## Year_Birth  -0.004914787  1.000000000 -0.142946630 -0.020745551
## Income      0.009816702 -0.142946630  1.000000000 -0.002345632
## Recency     0.005182121 -0.020745551 -0.002345632  1.000000000
## NumWebPurchases  0.003150136 -0.146032492  0.350084878 -0.007504121
## NumStorePurchases 0.003558618 -0.127863561  0.488805136 -0.002865018
## NumWebVisitsMonth 0.001880957  0.119387890 -0.505789042 -0.033195044
## Response      -0.006662268  0.010156677  0.131523893 -0.195565532
##              NumWebPurchases NumStorePurchases NumWebVisitsMonth
## ID          0.003150136      0.003558618      0.001880957
## Year_Birth  -0.146032492      -0.127863561      0.119387890
## Income      0.350084878      0.488805136      -0.505789042
## Recency     -0.007504121      -0.002865018      -0.033195044
## NumWebPurchases  1.000000000      0.511799518      -0.037556589
## NumStorePurchases 0.511799518      1.000000000      -0.434079557
## NumWebVisitsMonth -0.037556589      -0.434079557      1.000000000
## Response      0.162880277      0.040937927      -0.001195599
##              Response
## ID          -0.006662268
## Year_Birth   0.010156677
## Income       0.131523893
## Recency     -0.195565532
## NumWebPurchases  0.162880277
## NumStorePurchases 0.040937927
## NumWebVisitsMonth -0.001195599
## Response      1.000000000
```

```
corrplot(correlation, method="circle")
```

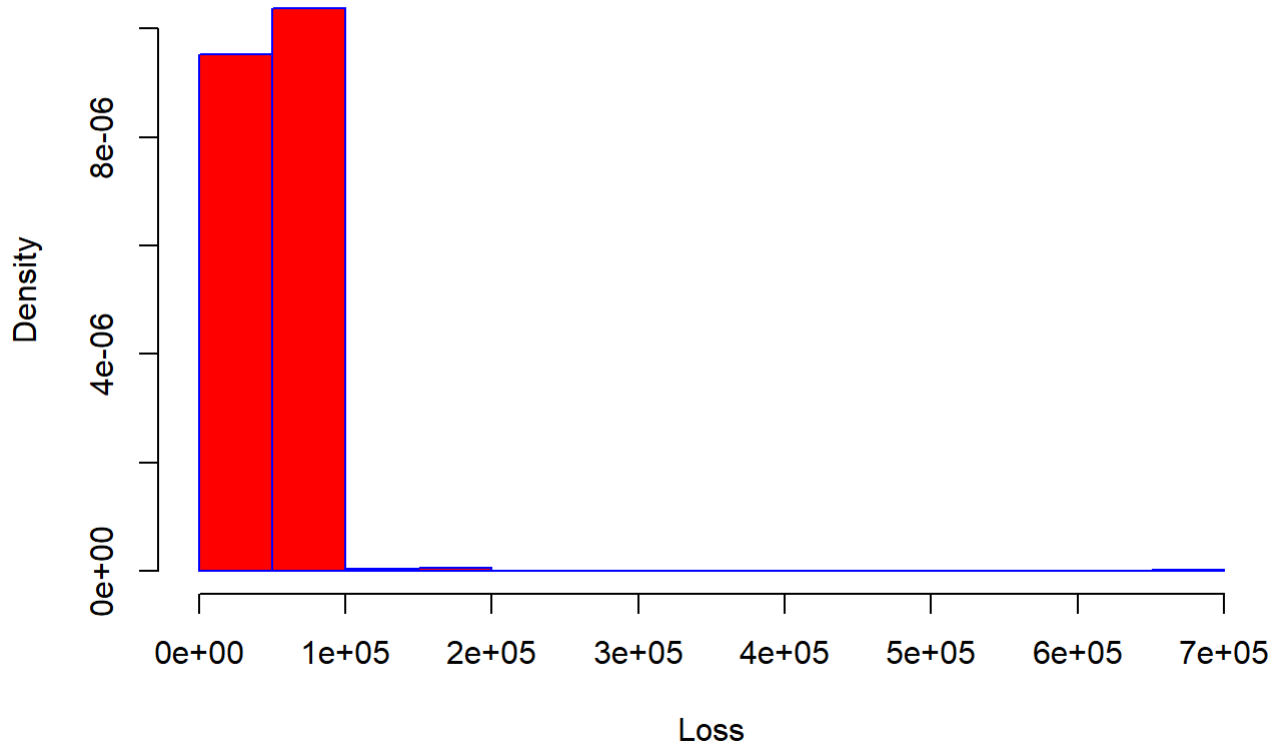


```
# check if there is outlier in income
boxplot(clean_data$Income)
```

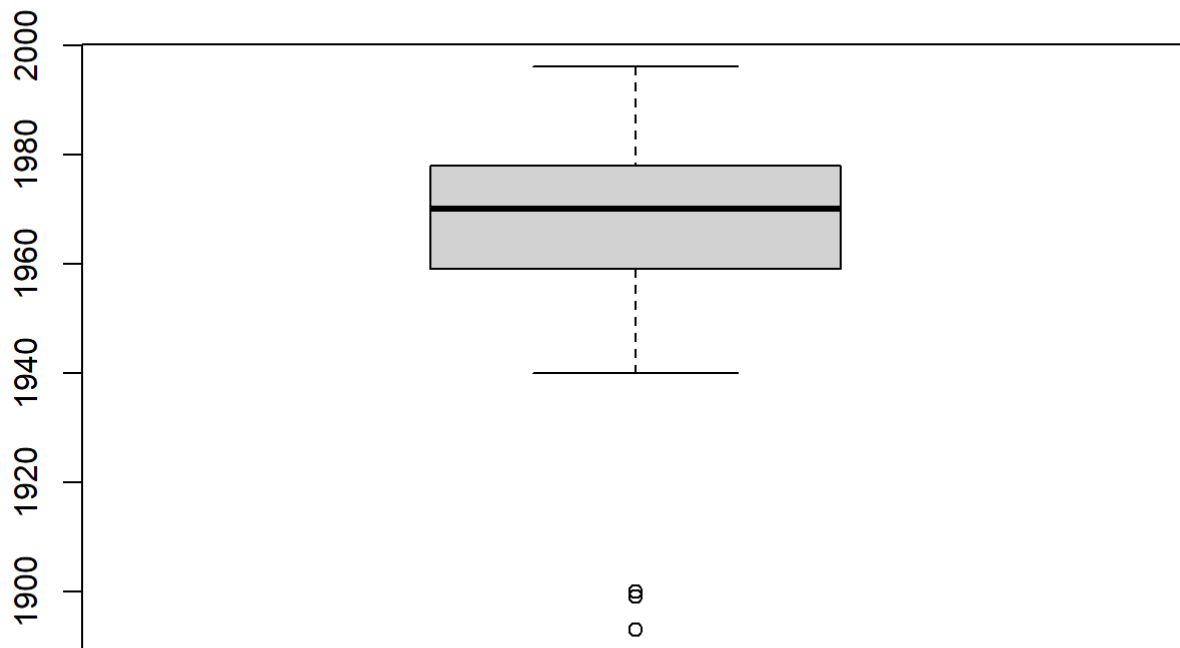


```
# checking for income distribution
hist(clean_data$Income,
      main="Income distribution ",
      xlab="Loss",
      border="blue",
      col="red",
      prob =TRUE)
```

Income distribution



```
#checking outlier in year of birth and level of income
boxplot(clean_data$Year_Birth)
```

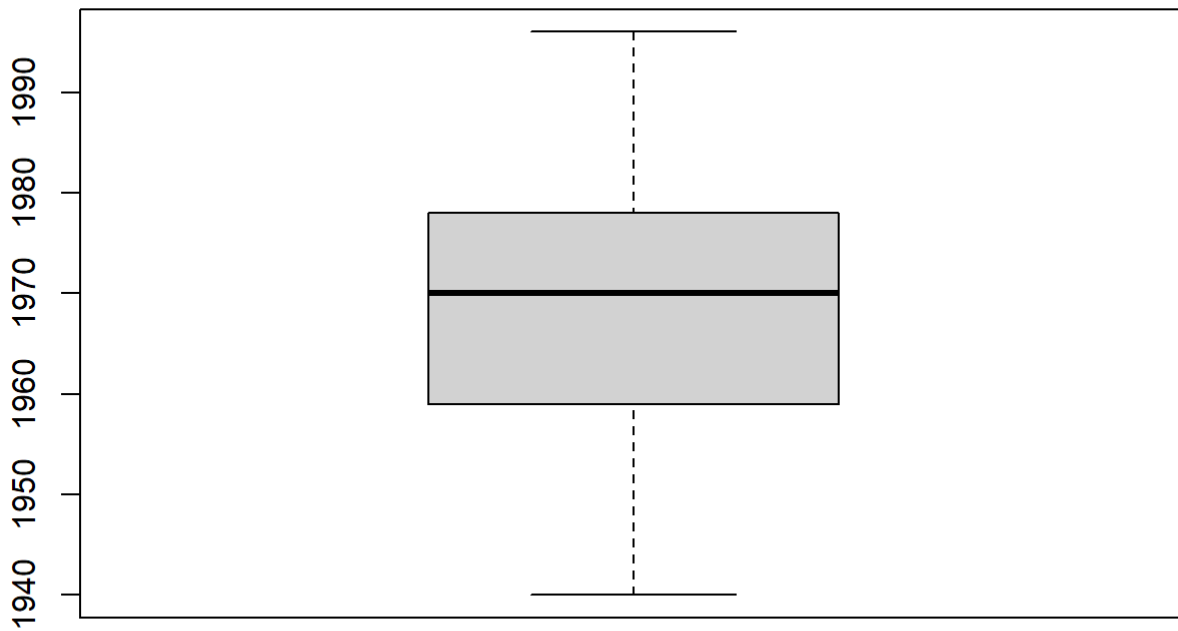


```
# fixing the outlier
# fixing outlier from the income column
remove_outlier <- function(x) {
  q <- quantile(x, probs = c(0.25, 0.75))
  iqr <- IQR(x)
  # lb = lower bound and ub = upper bound
  lb <- q[1] - 1.5 * iqr
  ub <- q[2] + 1.5 * iqr
  x[x < lb | x > ub] <- median(x, na.rm = TRUE)
  return(x)
}

# function call
clean_data$Income <- remove_outlier(clean_data$Income)
```

```
# for year of birth
clean_data <- clean_data[clean_data$Year_Birth > 1900, ]
```

```
# checking if outlier has been removed
boxplot(clean_data$Year_Birth)
```



1.3 Exploratory Data Analysis

To gain profound insights into the demographic characteristics of customers in order to facilitate precise predictions, a visualization analysis was conducted. Despite the majority of participants being graduates, a significant proportion of them (9672) declined the promotional campaign, followed by 3800 respondents holding a PhD, 3192 respondents with a master's degree, and 1833 respondents at the second cycle level. Notably, a large portion of the respondents are married, with 7605 of them who are married rejecting the campaign offer, followed by 5239 in relationships and 3755 who are single. Furthermore, it was determined that the duration since the customer's last purchase and the frequency of visits to the company's website in the previous month have no impact on the response to the campaign offer. It was observed that certain respondents had significantly higher incomes compared to others, potentially leading to a negative influence on their response to the campaign. Similarly, discrepancies were noted in the reported year of birth, with some respondents mistakenly indicating a birth year prior to 1900.

1.4 Feature Engineering

```
#Feature Engineering

# using dplyr to remove irrelevant column
clean_data <- dplyr::select(.data = clean_data, -ID)

#Encoding categorical variables.
clean_data$Education <- factor(clean_data$Education, levels = c("2n Cycle", "Basic", "Graduation", "Master", "PhD"),
                               labels = c(1,2,3,4,5)
                               )

clean_data$MaritalStatus <- factor(clean_data$MaritalStatus, levels = c("Absurd", "Alone", "Divorced", "Married", "Single", "Together", "Widow", "YOLO"),
                                   labels = c(1,2,3,4,5,6,7,8)
                                   )

# Convert 1, 0 to Yes, No
clean_data$Response <- ifelse(clean_data$Response == 1, "Yes", "No")

clean_data$Response <- factor(clean_data$Response)
```

The education and marital status levels were label encoded in preparation to making model prediction.

1.5 Model Prediction

1.5.1 Decision Tree

```
#Decision Tree

# Set the fixed state for the random number generation
set.seed(20)

# Create a sample vector of row indices
sample <- sample(nrow(clean_data), 0.8*nrow(clean_data))

# Split data into training and testing sets
train_data <- clean_data[sample,]
test_data <- clean_data[-sample,]

#checking reponse variable of both train and test
table(train_data$Response)
```

```
##
##      No   Yes
## 14941  2747
```

```
table(test_data$Response)
```

```
##
##      No   Yes
##  3777   646
```

```

# Model Building on train data set
class_mod1 = rpart(Response ~ ., data = train_data)

# Prediction on test data set for the class
class_pred_test = predict(class_mod1, test_data, type = 'class')

# Making Confusion matrix
caret::confusionMatrix(class_pred_test, test_data$Response, positive = "Yes")

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction   No  Yes
##           No 3715 490
##           Yes  62 156
##
##           Accuracy : 0.8752
##           95% CI : (0.8651, 0.8848)
## No Information Rate : 0.8539
## P-Value [Acc > NIR] : 2.444e-05
##
##           Kappa : 0.3103
##
## Mcnemar's Test P-Value : < 2.2e-16
##
##           Sensitivity : 0.24149
##           Specificity : 0.98358
##           Pos Pred Value : 0.71560
##           Neg Pred Value : 0.88347
##           Prevalence : 0.14605
##           Detection Rate : 0.03527
## Detection Prevalence : 0.04929
##           Balanced Accuracy : 0.61254
##
##           'Positive' Class : Yes
##

```

```

# ROC AUC prediction
tree_trainControl <- trainControl(method="cv", number=2, classProbs=TRUE,
                                   summaryFunction=twoClassSummary)

tree_train <- train(Response~., data=train_data, method="rpart", metric="ROC",
                    trControl=tree_trainControl)

tree_predictions <- predict(tree_train, newdata = test_data, type="prob")

tree_predicted_probs <- tree_predictions[, 2]

# replace the positive label with 1 and else 0
tree_positive_labels <- ifelse(test_data$Response == "Yes", 1, 0)

tree_roc_curve <- roc(tree_positive_labels, tree_predicted_probs)

```

```

## Setting levels: control = 0, case = 1

```

```

## Setting direction: controls < cases

```

```
tree_auc_value <- auc(tree_roc_curve)
print(tree_auc_value)
```

```
## Area under the curve: 0.6893
```

The data was partitioned utilizing an 80:20 ratio, with 80% of the datasets comprising the training data and the remaining 20% constituting the test dataset. The decision model accurately forecasts that the likelihood of a customer accepting or declining an offer stands at 87.52% based on the test data, in the event that there is a need to assess the model on unseen data.

In other words, sensitivity (tpr) and specificity (tnr) metrics can be more informative. The model's sensitivity was predicted at 2.41%, a notably low figure, suggesting inefficiency in predicting customers likely to respond positively to a campaign offer. Conversely, the model's specificity at 98.35% indicates strong performance in predicting customers unlikely to respond positively to the campaign offer. This suggests the model's effectiveness in predicting a customer's lack of active response to a campaign offer.

Given the prior observation of data imbalance, relying solely on accuracy metrics may be deceptive. Researchers have addressed this data imbalance by introducing various metrics for imbalanced data domains, including but not limited to area under the curve (AUC), Matthews correlation coefficient (MCC), G-mean, Kappa, and several others, a few of which have been effectively adapted for multi-class scenarios (Sun et al & Zhen et al). Moreover, employing ROC_AUC aids in enhancing data imbalance model's efficiency due to its insensitive to class distribution because of it being the trajectory of the false positive rate (FPR) and true positive rate (TPR) pairs. With an AUC of 68.93%, the model's ability to differentiate between customers with positive responses to the campaign and those without is reasonably satisfactory, surpassing the threshold of 0.5. Nevertheless, a 68.93% AUC suggests that the model surpasses random chance in predicting a response to a campaign offer.

1.5.2 Logistic Regression

```
#Logistic Regression
# Build logistic regression model
log_model <- glm(Response ~ ., data = train_data, family = binomial)
summary(log_model)
```

```
##
## Call:
## glm(formula = Response ~ ., family = binomial, data = train_data)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -3.1678  -0.5704  -0.3678  -0.2105   3.1641
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -3.998e+01  4.189e+00  -9.543  < 2e-16 ***
## Year_Birth    1.808e-02  2.100e-03   8.610  < 2e-16 ***
## Education2   -1.536e-02  3.051e-01  -0.050  0.95984
## Education3    4.475e-01  1.009e-01   4.437  9.13e-06 ***
## Education4    6.277e-01  1.105e-01   5.682  1.33e-08 ***
## Education5    1.015e+00  1.056e-01   9.606  < 2e-16 ***
## MaritalStatus2 1.164e-02  6.550e-01   0.018  0.98583
## MaritalStatus3 -5.729e-01  5.249e-01  -1.091  0.27506
## MaritalStatus4 -1.266e+00  5.223e-01  -2.423  0.01539 *
## MaritalStatus5 -2.727e-01  5.223e-01  -0.522  0.60163
## MaritalStatus6 -1.383e+00  5.232e-01  -2.643  0.00823 **
## MaritalStatus7 -1.157e-01  5.300e-01  -0.218  0.82724
## MaritalStatus8 -1.682e-01  7.998e-01  -0.210  0.83343
## Income         6.124e-05  2.135e-06  28.680  < 2e-16 ***
## Recency        -2.244e-02  8.587e-04 -26.130  < 2e-16 ***
## NumWebPurchases 7.738e-02  1.073e-02   7.214  5.42e-13 ***
## NumStorePurchases -1.718e-01  1.028e-02 -16.720  < 2e-16 ***
## NumWebVisitsMonth 2.332e-01  1.386e-02  16.822  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 15275  on 17687  degrees of freedom
## Residual deviance: 12446  on 17670  degrees of freedom
## AIC: 12482
##
## Number of Fisher Scoring iterations: 6
```

```
exp(coef(log_model))
```

```
##      (Intercept)      Year_Birth      Education2      Education3
## 4.354580e-18 1.018241e+00 9.847536e-01 1.564455e+00
##      Education4      Education5      MaritalStatus2      MaritalStatus3
## 1.873281e+00 2.758795e+00 1.011703e+00 5.638988e-01
##      MaritalStatus4      MaritalStatus5      MaritalStatus6      MaritalStatus7
## 2.820947e-01 7.613527e-01 2.509418e-01 8.907676e-01
##      MaritalStatus8      Income      Recency      NumWebPurchases
## 8.451828e-01 1.000061e+00 9.778124e-01 1.080448e+00
##      NumStorePurchases      NumWebVisitsMonth
## 8.421480e-01 1.262576e+00
```

```
train_data$Probs <- predict(object = log_model, data = train_data, type="response")
```

```
# checking if the prob has been added
head(train_data)
```

	Year_Birth	Education	MaritalStatus	Inco...	Rece...	NumWebPurcha...	NumStorePurchases	
	<int>	<fct>	<fct>	<int>	<int>	<int>	<int>	
17624	1974	3	5	69719	99	2	5	
18310	1969	5	5	48240	73	6	13	
1913	1968	3	5	36065	54	3	3	
5991	1971	3	4	25721	75	1	2	
4639	1973	5	4	37401	14	2	3	
21499	1982	3	6	71853	29	2	6	

6 rows | 1-8 of 11 columns

```
# making it a class prediction
train_data$ClassPredict <- ifelse(train_data$Probs > 0.5, "Yes", "No")

#Confusion matrix of results for Train
table(Actual = train_data$Response, Predicted = train_data$ClassPredict)
```

```
##           Predicted
## Actual    No    Yes
##    No 14620   321
##    Yes  2274   473
```

```
# percentage of correctness?
mean(train_data$ClassPredict == train_data$Response)
```

```
## [1] 0.8532904
```

```
# Make predictions on test data
test_data$Response <- factor(test_data$Response, levels = c("No", "Yes"))
test_data$prob <- predict(log_model, newdata = test_data, type = "response")
test_data$ClassPred <- ifelse(test_data$prob > 0.5, "Yes", "No")
test_data$ClassPred<- factor(test_data$ClassPred, levels = c("No", "Yes"))

# calculate the percentage of correct prediction
table(Actual=test_data$Response, Predicted = test_data$ClassPred)
```

```
##           Predicted
## Actual    No    Yes
##    No  3690   87
##    Yes   541  105
```

```
mean(test_data$ClassPred == test_data$Response)
```

```
## [1] 0.8580149
```

```
# Confusion matrix for Logistic regression
caret::confusionMatrix(test_data$ClassPred, test_data$Response, positive = "Yes")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No  Yes
##           No 3690 541
##           Yes  87 105
##
##           Accuracy : 0.858
##           95% CI : (0.8474, 0.8682)
##           No Information Rate : 0.8539
##           P-Value [Acc > NIR] : 0.2288
##
##           Kappa : 0.1968
##
##           Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.16254
##           Specificity : 0.97697
##           Pos Pred Value : 0.54687
##           Neg Pred Value : 0.87213
##           Prevalence : 0.14605
##           Detection Rate : 0.02374
##           Detection Prevalence : 0.04341
##           Balanced Accuracy : 0.56975
##
##           'Positive' Class : Yes
##
```

```
# preparing for resampling
log_trainControl <- trainControl(method="cv", number=2, classProbs=TRUE,
                                summaryFunction=twoClassSummary)

# ROC_AUC metrics

# re-assignment
train_data <- clean_data[sample,]
test_data <- clean_data[-sample,]

log_train <- train(Response~., data=train_data, method="glm", metric="ROC",
                  trControl=log_trainControl)

log_predictions <- predict(log_train, newdata = test_data, type="prob")

predicted_probs <- log_predictions[, 2]

# replace the positive label with 1 and else 0
positive_labels <- ifelse(test_data$Response == "Yes", 1, 0)

roc_curve <- roc(positive_labels, predicted_probs)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_value <- auc(roc_curve)
print(auc_value)
```

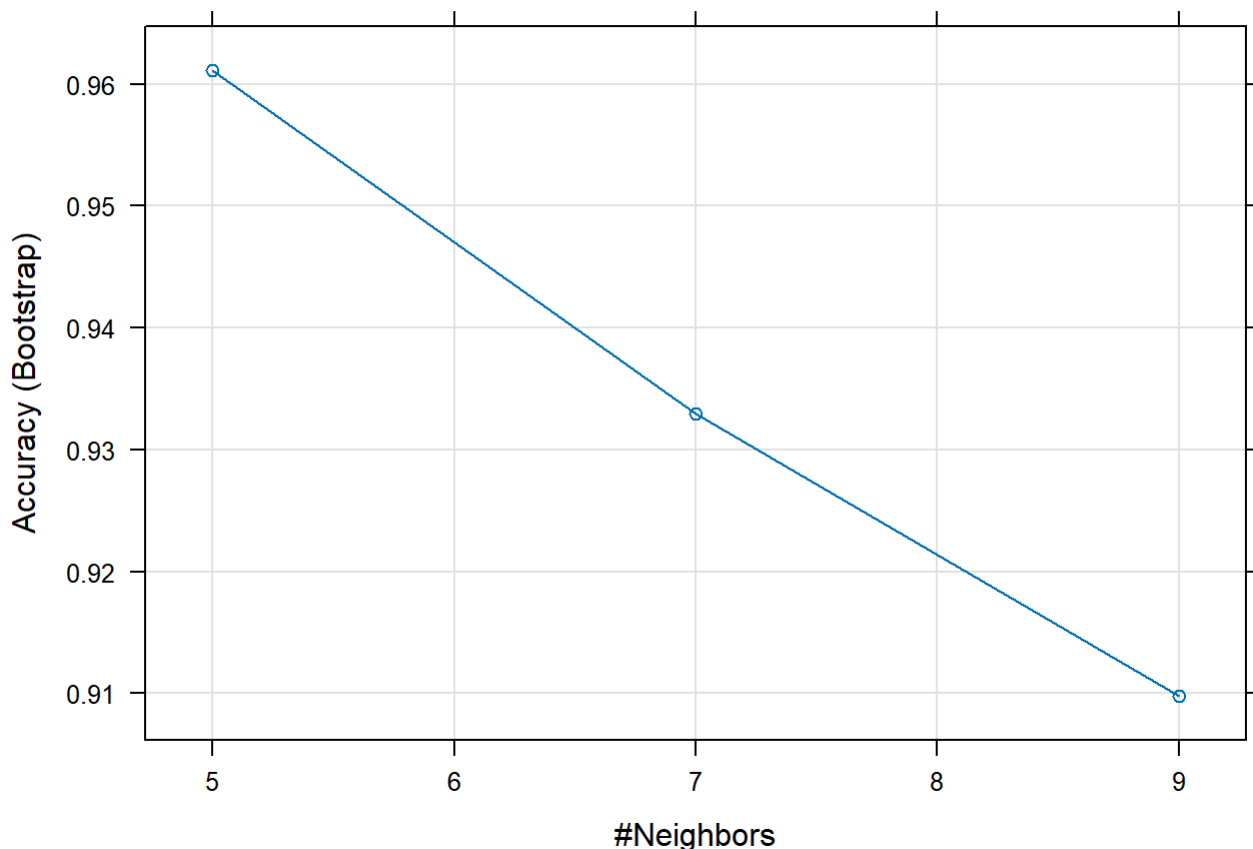
```
## Area under the curve: 0.7916
```

The summary of the logistic regression model revealed that an increase of one unit in variables such as year of birth, income, number of web purchases, number of web visits in the last month, and education results in a corresponding increase in the log odds of customer response to a campaign. The model accurately predicts that there is an 85.8% probability of a customer accepting or rejecting an offer based on the training data. Notably, the model also correctly predicts a 85.8% probability of a customer accepting or rejecting a campaign offer in the test data, which would be crucial for evaluating its performance on unseen data.

Given the imbalance in class distribution, it is imperative to avoid relying solely on accuracy metrics. Instead, metrics such as sensitivity (tpr) and specificity (tnr) should be considered. The model's sensitivity prediction of 1.63% indicates a low efficacy in identifying customers likely to respond positively to a campaign offer. Conversely, the model's high specificity of 97.70% suggests its effectiveness in identifying customers who are unlikely to respond positively. Furthermore, the utilization of ROC_AUC aids in better understanding the model. An AUC value of 79.16% signifies the model's satisfactory ability to differentiate between customers exhibiting positive responses to the campaign and those who do not. Essentially, a 79.16% AUC indicates that the model performs better than random chance in predicting responses to campaign offers, reflecting an enhancement compared to the decision ROC_AUC score metric.

1.5.3 KNN Model

```
#k Nearest Neighbours  
# Build KNN model  
  
knn_model <- train(form = Response ~.,  
                    data = train_data,  
                    method = 'knn')  
  
plot(knn_model)
```



```
# Making prediction on the test
knn_test_Pred <- predict(object= knn_model, newdata = test_data, type = "raw")
```

```
# checking the average prediction
mean(knn_test_Pred == test_data$Response)
```

```
## [1] 0.9771648
```

```
# Confusion matrix for KNN
confusionMatrix(data = knn_test_Pred, reference = test_data$Response)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No  Yes
##           No 3723  47
##           Yes  54 599
##
##           Accuracy : 0.9772
##           95% CI : (0.9723, 0.9814)
##           No Information Rate : 0.8539
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9089
##
## Mcnemar's Test P-Value : 0.5505
##
##           Sensitivity : 0.9857
##           Specificity : 0.9272
##           Pos Pred Value : 0.9875
##           Neg Pred Value : 0.9173
##           Prevalence : 0.8539
##           Detection Rate : 0.8417
##           Detection Prevalence : 0.8524
##           Balanced Accuracy : 0.9565
##
##           'Positive' Class : No
##
```

```
# ROC_AUC
# making prediction on the test
knn_pred <- predict(object = knn_model, newdata = test_data, type = "prob")
pred <- knn_pred[,2]
KnnROC <- roc(response = test_data$Response, predictor = pred)
```

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```
auc_value <- auc(KnnROC)
print(auc_value)
```

```
## Area under the curve: 0.9982
```

K-nearest neighbor represents the best model in prediction campaign offer with 97.6% accuracy. As mentioned earlier, we shouldn't solely rely on accuracy metric. It's important to consider Sensitivity, Specificity, and ROC_AUC for proper interpretation. With a sensitivity of 98.54%, the model excels at predicting customers who would respond positively to a campaign offer. The specificity of 92.11% indicates the likelihood of predicting customers who wouldn't respond positively. This aligns with balanced accuracy metrics showing a 95.55% chance of customer response to a campaign offer.

K-nearest neighbor AUC (99.82%) showcases the best model prediction. This high AUC value of 99.88% signifies the model's superior performance over random chance in predicting campaign offer responses. This illustrates that the KNN model can accurately and efficiently predict customer responses without any bias.

1.5.4 CLustering

```
#Clustering
new_data <- clean_data
#remove the response column
new_data$Response <- NULL

# preparing the following variables for scaling
features <- new_data[, c("Year_Birth", "Income", "NumWebVisitsMonth", "NumStorePurchases",
                        "NumWebPurchases")]

feature_means <- apply(features,2,mean)
feature_stds <- apply(features,2,sd)

scaled_features <- scale(features,center=feature_means,scale=feature_stds)

# using the dendrogram
# the distance
datadist <- dist(x = scaled_features, method = "euclidean")
hcdata <- hclust(d = datadist, method = "complete")

# create dendrogram using the reduced attribute sets
datadend <- as.dendrogram(hcdata)

#Levels of the campaign variable
campaignlevels <- rev(levels(clean_data[,9]))

# branches color based on clusters:
datadend <- color_branches(dend = datadend, k=3)

# assign one of the colours to each label ordered by the dendrogram based on campaign response,
labels_colors(datadend) <-
  rainbow_hcl(2)[sort_levels_values(
    as.numeric(clean_data[,9])[order.dendrogram(datadend)]
  )]

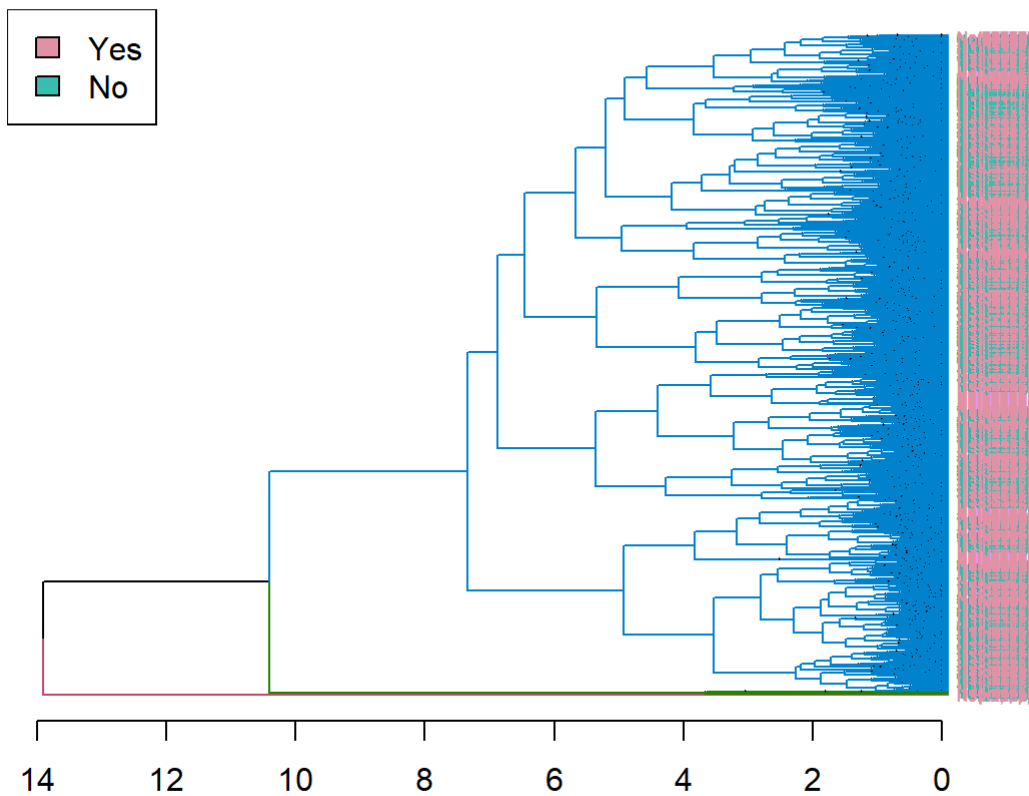
# combine the response to the labels: ordered by the dendrogram
labels(datadend) <- paste(as.character(clean_data[,9])[order.dendrogram(datadend)],
                        "(",labels(datadend),")",
                        sep = "")

# distance between end of dendrogram and the label
datadend <- hang.dendrogram(datadend,hang_height=0.1)

# label size reduction
datadend <- set(dend = datadend, what = "labels_cex",value = 0.5)

# And plot:
par(mar = c(3.5,2.75,3,7))
plot(datadend,
     main = "Natural grouping of Campaign Response",
     horiz = TRUE,
     nodePar = list(cex = .006))
legend("topleft", legend = campaignlevels, fill = rainbow_hcl(2))
```

Natural grouping of Campaign Response



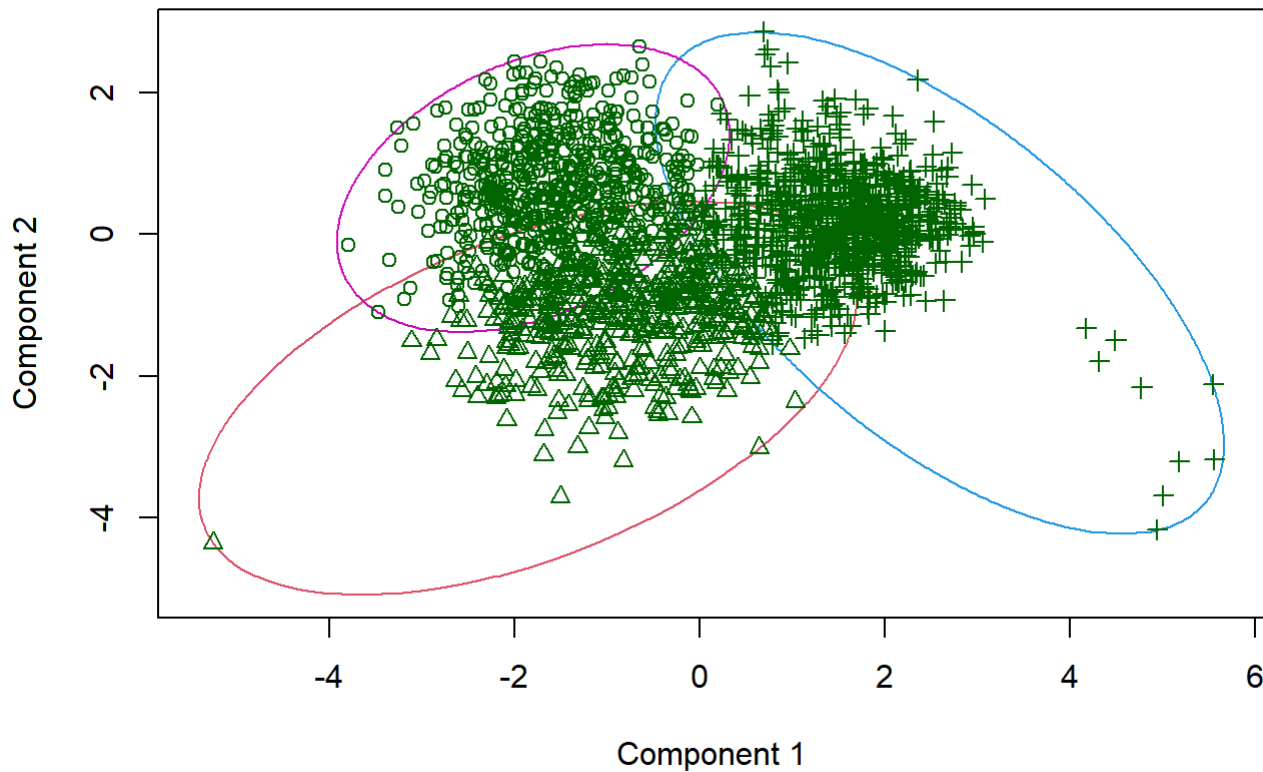
```
#Using the kmeans algorithm to determine relevant number of k
Kmeandata <- kmeans(x = scaled_features, center = 3, nstart = 5) # neat right!
```

```
# checking the cluster
table(Response = clean_data$Response, Cluster = Kmeandata$cluster)
```

```
##          Cluster
## Response    1    2    3
##      No  5238 4502 8978
##      Yes  1267 1207  919
```

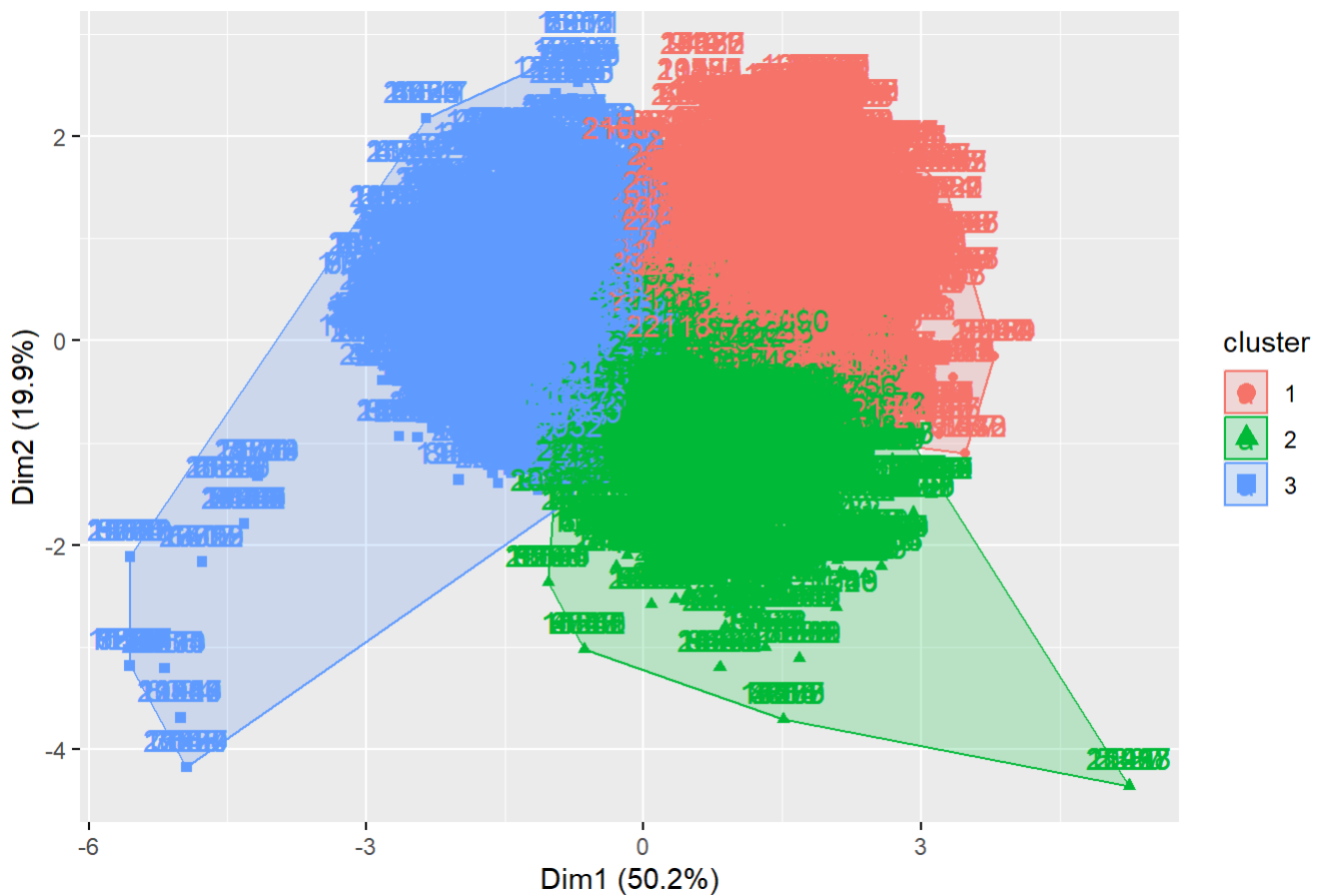
```
# kmean cluster viz
clusplot(scaled_features, Kmeandata$cluster, color = TRUE)
```

CLUSPLOT(scaled_features)



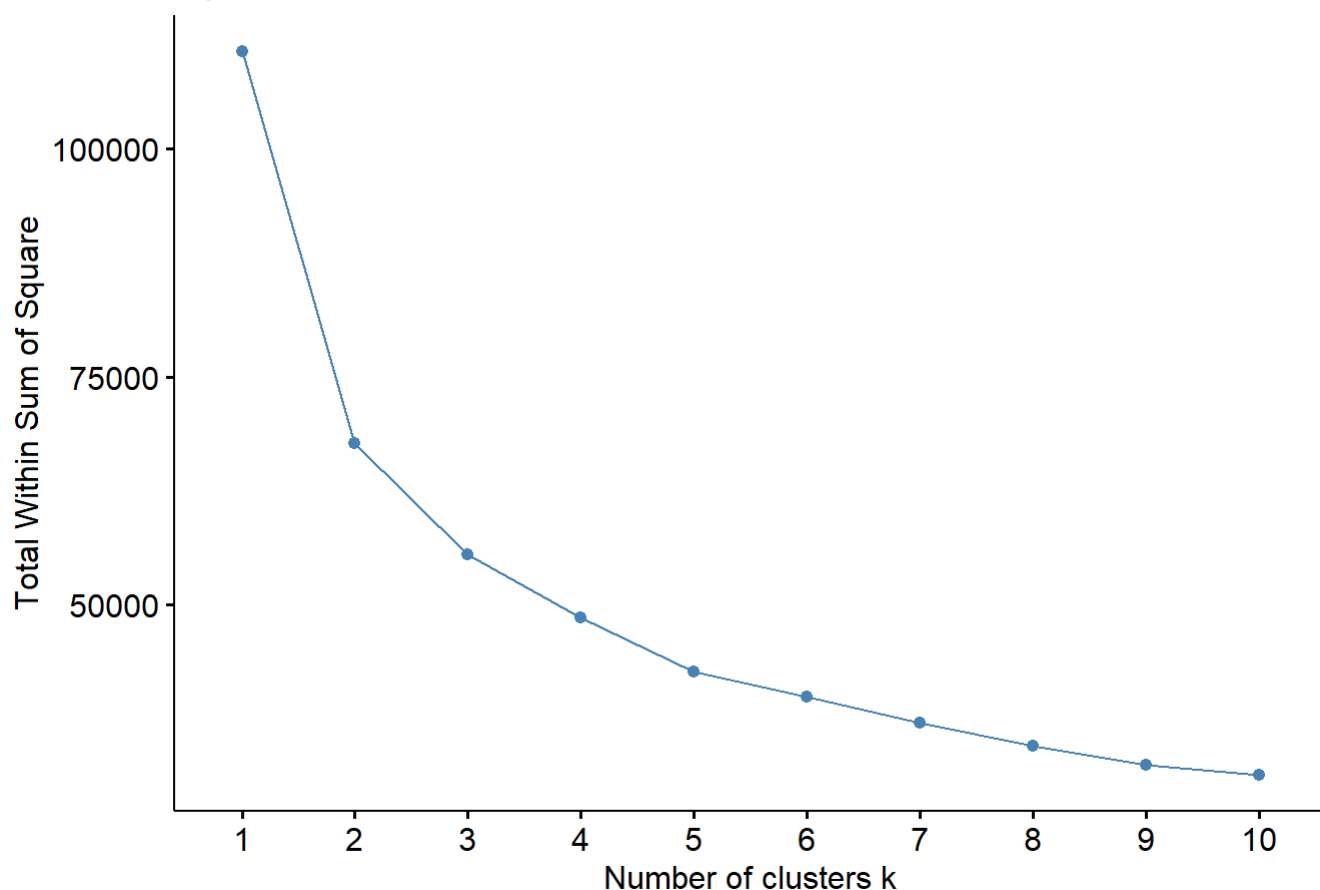
```
#illustration of Clusters
fviz_cluster(Kmeandata, data = scaled_features)
```

Cluster plot



```
#checking the optimal number of k
fviz_nbclust(scaled_features, kmeans, method = "wss")
```

Optimal number of clusters



```
#Determining Membership
member = cutree(hcdata,3)
table(member)
```

```
## member
##      1      2      3
## 21992    30     89
```

```
# mean of membership cluster
aggregate(scaled_features,list(member),mean)
```

Group.1<int>	Year_Birth<dbl>	Income<dbl>	NumWebVisitsMonth<dbl>	NumStorePurchases<dbl>	NumWebPurchas...<dbl>
1	0.0006806262	0.01068181	-0.01710786	0.009282846	-0.004511556
2	-0.5292337158	-1.16826300	-1.97686340	-1.656759839	7.413658433
3	0.0102098776	-2.24569018	4.89373099	-1.735343288	-1.384175383

3 rows

Customer segmentation plays a key role in marketing strategy, allowing businesses to customize their campaign approaches for different customer groups based on their unique traits and behaviors. To efficiently segment customers based on preferences for a particular offer, K-mean clustering is utilized. The first cluster reveals that most customers share a similar birth period, indicating a relatively uniform age group with slightly lower income compared to other clusters. It is interesting to note that customers in this group browse the website, visit the store, and make online purchases at similar times. This cluster represents an average market segment with typical buying habits.

In the second cluster, customers were born earlier than those in Cluster 1 and have lower income compared to other clusters. Additionally, they visit the website less frequently. While they make fewer in-store purchases than Cluster 1, their online purchases exceed those in the first cluster. Hence, marketing strategies should focus on promoting online activities

and providing exclusive web-based offers to better engage this segment.

Individuals in the third cluster are born later and have higher income compared to Clusters 1 and 2. They visit the website more often than Cluster 1 but less than Cluster 2. On average, customers in this group make fewer in-store and online purchases compared to Clusters 1 and 2. This suggests that these customers are affluent, older individuals who interact with the brand online but may need incentives or personalized offers to convert their browsing behavior into actual purchases. Marketing initiatives could highlight the brand's value proposition and offer customized deals or discounts to encourage conversion.

1.6 Conclusion

From the decision tree, logistic regression model to K-nearest neighbor (KNN), it can be deduced that KNN emerges as the optimal predictive model. Hence, KNN is proficient in predicting the likelihood of a customer accepting or declining a campaign offer by leveraging the distinct attributes within the dataset based on the ROC AUC metric due to data imbalance. According to Obuchowski & Bullen (2018), Receiver Operating Characteristic (ROC) analysis, comprising ROC plots and standard metrics, serves as the conventional method for model selection and assessment in the context of two-class classification issues. Accuracy metric cannot be used due to bias, hence the reason for introducing ROC. However, the examination of marketing campaign offers valuable insights for the optimization of marketing strategies. Through a comprehensive comprehension of the unique preferences and behaviors exhibited by various customer segments, enterprises can customize their approaches to effectively target and engage with each distinct group. This focused strategy contributes to the enhancement of customer satisfaction, an increase in conversion rates, and ultimately, a rise in revenue and profitability.

1.7 Recommendations:

In accordance with the outcomes of the campaign analysis, these recommendations are suggested:

- Execution of targeted marketing campaigns specifically designed to align with the distinct preferences of each customer segment.
- Augmentation of promotions online and incentivise them to further captivate tech-savvy customers. Creation of personalized offers and promotions aimed at converting browsing activity into purchases among affluent customers that are older.
- Incorporation of supplementary data sources to heighten the accuracy of segmentation. Continuous monitoring and assessment of marketing strategies to adapt to evolving customer preferences and market trends.

References

- 1. Sun Y, Kamel MS, Wang Y. Boosting for learning multiple classes with imbalanced class distribution. In: Sixth international conference on data mining (ICDM'06); 2006. p. 592–602. Return to ref 19 in article
- 2. Zhen L, Qiong L. A new feature selection method for internet traffic classification using ml. Phys Procedia. 2012;1(33):1338–455.
- 3. N. A. Obuchowski and J. A. Bullen, "Receiver operating characteristic (ROC) curves: Review of methods with applications in diagnostic medicine", Phys. Med. Biol., vol. 63, no. 7, 2018.

-4. H. Lee, C.-H. Cho Digital advertising: Present and future prospects International Journal of Advertising, 39 (3) (2019), pp. 1-10.

-5. S. Wilkes, C. Chapman, L. McCall Influential article review - on instagram, endorsers' digital marketing campaigns Journal of Marketing Development and Competitiveness, 15 (4) (2021).